



5 PRODUCTION AI PROJECTS FOR RESUMES



- ✓ Stop building toy wrappers. Build real production systems.
- ✓ 5 Systems that get callbacks + **3 to delete immediately** .

5 REAL PROJECTS
PRODUCTION GRADE

3 CLICHÉS
DELETE TODAY

EXACT BULLETS
RESUME READY

Swipe to upgrade your AI Portfolio



The Harsh Reality: Why 90% of AI Resumes Get Rejected

 The "Tutorial Clone" Basic Jupyter Notebook calling OpenAI API with zero evaluation or latency bounds.	 Production System Containerized API with Hybrid RAG, LangGraph agents, PII guardrails & metrics.
---	---

What Hiring Managers Actually Look For:

- **Software Engineering Rigor:** Clean Python, Pydantic v2 typing, Dockerfiles, and CI/CD tests.
- **Latency & Cost Awareness:** Cross-Encoder reranking, prompt caching, and token budget calculations.
- **Evaluation & Reliability:** Automated benchmarks using Ragas / LLM-as-a-Judge instead of "vibes".
- **Real Scale Architecture:** Decoupled ingestion queues, connection pools & error retries.

 Recruiter Stat: The Solution:	Technical recruiters spend only 6 seconds scanning projects before deciding on an interview callback. Showcase production trade-offs, quantifiable metrics, and deployable Docker microservices.
--	--

Delete Immediately: 3 Cliché Projects Hurting Your Resume



1. Titanic Kaggle



2. Iris Flower



3. 10-line ChatGPT Wrapper

Why These Hurt Your Candidacy:

- **1. Titanic & Iris Classification:** Signals that you only copied a 2017 beginner tutorial and never built production software.
- **2. Basic ChatGPT Clones:** Writing 10 lines of ``openai.chat.completions`` with a Streamlit UI shows zero engineering depth.
- **3. MNIST Digit Recognizer:** Academic toy problem that demonstrates zero knowledge of modern LLM systems.
- **The Fix:** Replace toy tutorials with systems handling real database writes, state persistence & security.

 **Action Item:** Replace these 3 cliché projects with the 5 production systems on the following slides.

 **Warning:** Listing toy projects signals to senior engineers that you lack practical production experience.

Project 1: Enterprise Financial Document Hybrid RAG



Naive RAG fails on financial PDFs with complex tables and exact SKU codes. This system provides 100% grounded answers.

Architecture Highlights:

- **Table-Aware Chunking:** Parses Markdown tables as JSON structures to preserve row/col headers.
- **Hybrid Retrieval (RRF):** Dense Vector Search (Qdrant) + Sparse BM25 Keywords for ticker symbols.
- **Two-Stage Reranking:** Cross-Encoder evaluates top 50 chunks down to top 5 pristine chunks.
- **Clickable Page Attribution:** Highlights source lines directly in the PDF viewer UI.

```
</> INGESTION PIPELINE LOGIC:  
ExtractTables() -> ChunkOverlap(15%) -> Embed(text-3-large) -> Qdrant.Upsert()
```

Impact: Cuts hallucination rate from 28% down to <1.2% while providing exact page/line citations.

Project 1: Tech Stack & Exact Resume Bullets

Recommended Tech Stack:

Python 3.12

Qdrant Vector DB

Cohere Rerank v3

FastAPI + React

COPY-PASTE RESUME BULLET:

"Architected an Enterprise Financial RAG pipeline over 500+ SEC 10-K filings using Hybrid Search (BM25 + Qdrant) and Cohere Reranking, improving retrieval precision by 42% and eliminating hallucinations via citation highlighting."

Interviewer Talking Points:

- Explain how Reciprocal Rank Fusion (RRF) combines sparse keyword scores with dense embedding similarities.
- Discuss how Cross-Encoder reranking eliminates token waste in downstream LLM context windows.
- Explain how parent-document retrieval links small search chunks back to full contextual sections.



**Real-World
Analog:**

Morgan Stanley & Bloomberg production internal document retrieval stacks.

Project 2: Autonomous Support Agent with LangGraph



Single prompt agents crash in loops. This system uses deterministic state machines with tool calling and human gates.

Architecture Highlights:

- **LangGraph State Machine:** Graph workflow with conditional routing and cycle limits (max 5 steps).
- **Redis Session Checkpointer:** Persists multi-turn conversation state & thread history.
- **Human-in-the-Loop (HITL):** Sensitive actions (refunds > \$50) pause for human agent authorization.
- **Pydantic Argument Validation:** Strictly validates all JSON parameters before invoking DB tools.

 **STATE GRAPH ROUTING:**
UserQuery → ClassifyIntent → [FetchOrder | IssueRefund | EscalateHuman]

 **Reliability:** LangGraph state machines prevent agent hallucination loops and ensure 100% deterministic tool execution.

Project 2: Tech Stack & Exact Resume Bullets

Recommended Tech Stack:

LangGraph

Pydantic Tool
Calling

Redis Memory

PostgreSQL

COPY-PASTE RESUME BULLET:

"Engineered a multi-agent customer support system with LangGraph state machines and Redis checkpoints, executing dynamic SQL tool calls with Human-in-the-Loop gating, resolving 73% of tier-1 support tickets autonomously."

Interviewer Talking Points:

- Describe how state graphs prevent infinite recursive loops and how Redis manages thread persistence across user restarts.
- Explain how OAuth token scopes restrict tools to read-only queries by default.
- Discuss handling API rate-limits and exponential backoff retry strategies for external microservices.



Real-World
Analog:

Klarna Customer Service Agent resolving 2M support cases monthly.

Project 3: Self-Hosted Production LLM Serving API



Moving beyond third-party APIs: Deploying and benchmarking private, cost-effective open-source LLMs.

Architecture Highlights:

- **Continuous Batching & PagedAttention:** Uses vLLM to saturate GPU tensor cores without memory fragmentation.
- **INT4 AWQ Quantization:** Serves Llama-3-8B in under 6GB VRAM with sub-25ms Time-To-First-Token.
- **Prometheus & Grafana:** Live monitoring of TTFT, token throughput (tokens/sec), and GPU VRAM utilization.
- **Docker & CUDA:** Reproducible container deployment with automated GPU health checks.

VLLM PRODUCTION LAUNCH COMMAND:

```
vllm serve meta-llama/Llama-3-8B-Instruct --quantization awq --gpu-memory-utilization 0.90
```



Cost

Efficiency:

Self-hosting open-source LLMs cuts API inference bills by up to 85% at scale.

Project 3: Tech Stack & Exact Resume Bullets

Recommended Tech Stack:

vLLM

Docker & CUDA

Prometheus /
Grafana

FastAPI Gateway

COPY-PASTE RESUME BULLET:

"Deployed a high-throughput LLM inference microservice using vLLM PagedAttention and AWQ 4-bit quantization, scaling Llama-3 to 110 tokens/sec across 25 concurrent streams while monitoring latency via Prometheus."

Interviewer Talking Points:

- Demonstrate how PagedAttention solves the virtual memory fragmentation problem in GPU KV Cache allocation.
- Explain the latency vs throughput trade-off during continuous batching saturation.
- Discuss load balancing multiple vLLM instances with NGINX round-robin and health checks.

 **Real-World Analog:** Anyscale, Together AI, and internal inference infrastructure at scale.

Project 4: Enterprise AI Guardrails & PII Masking Gateway



AI Security is mandatory in healthcare and finance. This proxy sits before the LLM to filter attacks and scrub PII.

Architecture Highlights:

- **Ingress Injection Defense:** Llama Guard 3 filters hostile jailbreak attempts in <15ms.
- **Egress PII Masking:** Microsoft Presidio anonymizes SSNs, credit cards, and HIPAA PII via Named Entity Recognition.
- **Canary Token Leakage Check:** Drops response if internal system prompts are exfiltrated.
- **Zero Overhead Reverse Proxy:** FastAPI middleware adds <20ms overhead to total roundtrip latency.

 **GUARDRAIL MIDDLEWARE FLOW:**
Inspect(Input) → FilterInj(LlamaGuard) → Anonymize(Presidio) → StreamToLLM()

 **Security Standard:** Essential for SOC2 and HIPAA compliance in enterprise generative AI deployments.

Project 4: Tech Stack & Exact Resume Bullets

Recommended Tech Stack:

Microsoft
Presidio

Llama Guard 3

NeMo Guardrails

FastAPI Reverse
Proxy

COPY-PASTE RESUME BULLET:

"Built an enterprise AI Guardrail proxy intercepting 10k+ daily prompts, implementing real-time PII anonymization via Microsoft Presidio and automated prompt injection defense via Llama Guard 3, achieving 99.4% safety compliance."

Interviewer Talking Points:

- Explain how you balance guardrail latency overhead (<20ms) against security compliance requirements.
- Discuss how regex entropy scans catch AWS keys and JWT tokens in streaming egress.
- Detail how semantic similarity clustering detects novel zero-day jailbreak jailbreak variations.

 **Real-World Analog:** Guardrails AI & NVIDIA NeMo enterprise safety gateways.

Project 5: Automated LLM Evaluation CI/CD Pipeline



Never deploy prompt or model changes blindly: Automated regression testing for generative AI systems.

Architecture Highlights:

- **Ragas Triad Metrics:** Evaluates Context Relevance, Groundedness (Faithfulness), and Answer Relevance.
- **GitHub Actions CI/CD Gate:** Automatically blocks PR merging if groundedness score drops below 0.90.
- **Synthetic Dataset Generation:** Generates 200 gold-standard test queries from raw documents automatically.
- **Cost-Effective Judge:** Uses GPT-4o-mini to run evaluations asynchronously at \$0.05 per PR.

CI/CD GITHUB ACTION STEP:

```
pytest tests/eval.py --ragas-threshold=0.92 --fail-on-regression
```

Quality Assurance:

Shows hiring teams you understand quantitative quality assurance in stochastic AI systems.

Project 5: Tech Stack & Exact Resume Bullets

Recommended Tech Stack:

Ragas / TruLens

GitHub Actions

LLM-as-a-Judge

Pytest

COPY-PASTE RESUME BULLET:

"Implemented an automated LLM regression testing pipeline in GitHub Actions using Ragas and synthetic evaluation datasets, gating production deployments on Groundedness (>0.92) and reducing regression bugs by 65%."

Interviewer Talking Points:

- Explain how to mitigate position bias in LLM-as-a-Judge by swapping candidate order during pairwise evaluation.
- Discuss how automated synthetic test generation catches edge cases humans overlook.
- Explain tracking evaluation scores across commits with historical dashboards.



**Real-World
Analog:**

Continuous model CI/CD pipelines used at OpenAI, Anthropic, and Scale AI.

GitHub Polish: How to Structure Your Repository for Recruiters

✗ Messy Repo (Ignored)

Single `.ipynb`` file with no README, hardcoded API keys & no setup instructions.

✓ Pro Repo (Hired)

Architecture GIF diagram, ``docker-compose up`` 1-click run, typed tests & live demo URL.

Must-Have Elements in Every Project Repo:

- **1. Visual Architecture Diagram:** High-res ASCII or Mermaid diagram at top of README.
- **2. 1-Click Reproducibility:** ``docker compose up`` command that works out of the box.
- **3. Live Demo URL:** Deployed on Streamlit Cloud / Modal / Render with free access.
- **4. Clean `.env.example``:** Never commit secrets or API tokens to public Git history.

★ **Recruiter Tip:** A repository with automated GitHub Actions badges and 1-click Docker launch immediately signals senior-level hygiene.

Interview Defense: **How to Talk About Your Projects in Interviews**

Structure every project explanation using the **STAR-AI Framework** to stand out from 99% of candidates.

1. Situation & Scale:

- "I built a Hybrid RAG pipeline designed to handle 500+ SEC financial filings with sub-200ms latency."

2. Trade-offs & Engineering Decisions:

- "Vector search alone missed exact stock tickers (e.g. \$NVDA), so I introduced BM25 sparse search with Reciprocal Rank Fusion."

3. Measured Outcomes:

- "This improved retrieval precision by 42% and reduced evaluation hallucinations to under 1.2% measured via Ragas."

🕒 **Pro Tip:** Always quantify latency (ms), cost savings (%), and accuracy gains in your answers.

The 5–Project Portfolio: Full Tech Stack Overview

- 1. Enterprise Hybrid RAG: Qdrant + BM25 + Cohere Rerank + Citations.
- 2. LangGraph Support Agent: State Machine + Tool Calling + Redis Memory + HITL.
- 3. Self–Hosted LLM API: vLLM PagedAttention + Llama–3 AWQ + Docker.
- 4. AI Guardrail Gateway: Presidio PII Masking + Llama Guard 3 Injection Filter.
- 5. Automated CI/CD Evaluation: Ragas + GitHub Actions Regression Gating.

★ Hiring Signal: Having even 2 of these 5 projects on your resume puts you in the top 5% of all AI applicant pools.

📁 Result: Moves you from "junior prompt engineer" to "production AI systems architect".

LAND YOUR AI ROLE LIKE A PRO

Stop building toy wrappers. Build production-grade systems.

WHAT WE COVERED:

- 3 Cliché Projects to delete from your resume
- 5 Production AI systems with exact tech stacks & bullet points
- GitHub repo setup & interview defense strategies

Save & Repost to upgrade your AI resume!
Follow Resnur AI for daily AI engineering breakdowns.